Towards a General Logic Engine on Propagators

Lattice-Theoretic Foundations and Architectural Recommendations for Prologos

Prologos Project

2026-02-24

Contents

1 Executive Summary **3**

2 Part I: Lattice Foundations **4**
2.1 1. Why Lattices? 4
2.2 2. Crash Course: The Lattice Zoo 5
2.2.1 2.1 Partial Orders 5
2.2.2 2.2 Join-Semilattices 5
2.2.3 2.3 Complete Lattices and Top 5
2.2.4 2.4 Fixed Points: Why Lattices Guarantee Convergence 6
2.2.5 2.5 Finite Height and Termination 6
2.3 3. Key Lattice Structures for Logic Programming 6
2.3.1 3.1 The Substitution Lattice 6
2.3.2 3.2 The Herbrand Interpretation Lattice 7
2.3.3 3.3 The Interval Lattice 7
2.3.4 3.4 The Protocol State Lattice (Session Types) 7
2.3.5 3.5 The Multiplicity Semiring (QTT) 8

3 Part II: The Architectural Recommendation **8**
3.1 4. The Three-Layer Architecture 8
3.1.1 Layer 1: The Propagator Network (Monotonic Data Plane) 8
3.1.2 Layer 2: ATMS / Truth Maintenance (Non-Monotonic Control Plane) 9
3.1.3 Layer 3: Stratification (Controlled Non-Monotonicity) 9
3.2 5. Validating the Multiverse Mechanism 10

3.2.1 Why ATMS over Physical Forking 10
3.2.2 The Multiverse in Practice: A Worked Example 11
3.3 6. Validating the Pocket Universe Mechanism 12
3.3.1 Applications in the Logic Engine 12
3.4 7. Recovering SLD/SLG Semantics 13
3.4.1 7.1 SLD Resolution on Propagators 13
3.4.2 7.2 SLG Resolution (Tabling) on Propagators 13
3.4.3 7.3 Cut and Pruning 14
3.5 8. Tabling: The Completeness Mechanism 14
3.5.1 8.1 Design for Prologos 14
3.5.2 8.2 Lattice Answer Modes 15
3.5.3 8.3 Tabling as Pocket Universe 15
3.6 9. Connecting to Existing Prologos Infrastructure 16
3.6.1 9.1 The Elaborator as a Propagator Network (Today) 16
3.6.2 9.2 Session Types as Protocol Propagation 16
3.6.3 9.3 QTT Multiplicity Checking as Propagation 17

4 Part III: The Concrete Architecture 17
4.1 10. Core Data Types 17
4.1.1 10.1 Lattice Trait 17
4.1.2 10.2 Propagator Cell 17
4.1.3 10.3 Supported Values (ATMS Integration) 18
4.1.4 10.4 LVar (Lattice Variable) 18
4.2 11. The Propagator Network Runtime 18
4.2.1 11.1 Scheduler 18
4.2.2 11.2 Contradiction Handler 19
4.2.3 11.3 The `amb` Operator 19
4.2.4 11.4 Answer Collection 20
4.3 12. Phased Implementation Plan 20
4.3.1 Phase 0: Lattice Trait + Basic Cells (Foundation) . . 20
4.3.2 Phase 1: Elaborator Propagators (Compile-Time Use) 21
4.3.3 Phase 2: LVars + Tabling (Runtime Foundation) . . . 21
4.3.4 Phase 3: ATMS Layer (Nondeterministic Search) . . . 22
4.3.5 Phase 4: Stratified Evaluation (Negation + Aggregation) 22
4.3.6 Phase 5: Surface Syntax + Integration (User-Facing Logic Programming) 23
4.3.7 Phase 6: Galois Connections + Domain Embeddings (Advanced) 23

5 Part IV: Critique and Open Questions **23**
5.1 13. What This Architecture Does Well 23
5.2 14. What This Architecture Doesn't Solve (Yet) 24
5.3 15. Open Research Questions 25

6 Summary of Recommendations **25**

7 References **26**
7.1 Foundational (Must-Read) 26
7.2 Tabling and Logic Programming 26
7.3 Lattice-Based Languages 26
7.4 Theoretical Foundations 27
7.5 Constraint Systems 27
7.6 Type Systems 27
7.7 Implementations 27

1 Executive Summary

This document synthesizes two extensive research surveys — the Lattice-Based Propagator Networks survey and the Propagator Networks research report — into a concrete architectural recommendation for Prologos's logic engine. The central thesis:

> Prologos should build its logic engine as a **stratified propagator network** with an ATMS worldview layer, using lattice-compatible data structures (LVars) as the monotonic data plane and assumption-based truth maintenance as the non-monotonic control plane. Tabling provides completeness; stratification provides negation. The same substrate should serve both compile-time (type checking, elaboration) and runtime (logic programming, constraint solving) use cases.

The two user-proposed mechanisms — the "Multiverse Mechanism" (choice-point forking via ATMS worldviews) and the "Pocket Universe Mechanism" (scoped computation via lattice embeddings) — are both **validated** by the research and directly implementable. The Multiverse Mechanism maps onto ATMS's simultaneous worldview maintenance, recovering nondeterminism atop a monotonic substrate. The Pocket Universe Mechanism maps onto Galois connections between abstract and concrete domains, enabling modular constraint domains and scoped sub-computations.

Key architectural choices:

1. **Propagator cells** with user-defined lattice merge (Radul-Sussman model)
2. **ATMS layer** for hypothetical reasoning, `amb`, and dependency-directed backtracking
3. **Tabling** for termination and completeness (SLG-style, with lattice answer modes)
4. **Stratified evaluation** for negation, aggregation, and non-monotonic operations
5. **Galois-connected domain embeddings** for modular constraint solvers
6. **Phased implementation** starting with compile-time use (elaborator propagators), then extending to runtime logic programming

2 Part I: Lattice Foundations

2.1 1. Why Lattices?

A **lattice** is an algebraic structure that captures the notion of *partial information* and *refinement*. In programming language theory, lattices appear everywhere:

- **Type inference**: the substitution lattice (from most general to most specific)
- **Abstract interpretation**: concrete/abstract domain pairs connected by Galois connections
- **Constraint propagation**: cell values that accumulate knowledge monotonically
- **Distributed systems**: CRDTs converging through lattice joins

For a logic engine, lattices provide the mathematical guarantee that computation **converges** — that iteratively refining partial answers eventually reaches a stable fixed point.

2.2 2. Crash Course: The Lattice Zoo

2.2.1 2.1 Partial Orders

A **partial order** $(S, \leq)$ is a set S with a binary relation $\leq$ that is:

- **Reflexive**: $a \leq a$
- **Antisymmetric**: $a \leq b$ and $b \leq a$ implies $a = b$
- **Transitive**: $a \leq b$ and $b \leq c$ implies $a \leq c$

The ordering represents "has at least as much information as." In a type inference context: `Int` $\leq$ `?a` means "knowing the type is `Int` is more informative than knowing nothing (metavariable `?a`)."

Not all elements need be comparable — this is the "partial" in partial order. Two type constraints might be independent (neither implies the other).

2.2.2 2.2 Join-Semilattices

A **join-semilattice** $(L, \leq, \bot, \sqcup)$ adds:

- A **bottom** element $\bot$ (least informative — "nothing")
- A **join** operation $\sqcup$ (least upper bound — "combine information")

The join is:

- **Commutative**: $a \sqcup b = b \sqcup a$
- **Associative**: $(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c)$
- **Idempotent**: $a \sqcup a = a$

Join is the *merge* operation in propagator networks. When two propagators contribute information to the same cell, join combines them. Commutativity means the order doesn't matter; idempotency means duplicate contributions are harmless.

2.2.3 2.3 Complete Lattices and Top

A **complete lattice** also has:

- A **top** element $\top$ (maximally informative — *contradiction*)

- **Meet** operation $\sqcap$ (greatest lower bound)

Top represents incompatible information merged together. When a type checker discovers that a variable must be both `Int` and `String`, the result is $\top$ — contradiction. The propagator network must handle this by backtracking or reporting an error.

2.2.4 2.4 Fixed Points: Why Lattices Guarantee Convergence

Knaster-Tarski theorem: Every monotone function $f : L \to L$ on a complete lattice has a least fixed point:

$\text{lfp}(f) = \{x \in L \mid f(x) \leq x\}$

Kleene iteration: On a dcpo with $\bot$, the least fixed point of a Scott-continuous f is computed by iterating from bottom:

$\text{lfp}(f) = \bigsqcup_{n \in \mathbb{N}} f^n(\bot)$

This is exactly what propagator networks do: start with all cells at $\bot$ (no information), iterate propagators (apply f), and converge to the least fixed point (the most general solution).

2.2.5 2.5 Finite Height and Termination

If the lattice has **finite height** (longest ascending chain is bounded), then Kleene iteration terminates in at most h steps, where h is the height. For infinite-height lattices (e.g., intervals over reals), **widening** operators accelerate convergence:

$a \nabla b \geq a \sqcup b$

Widening over-approximates to ensure termination; **narrowing** then recovers precision:

$a \leq a \triangle b \leq a \quad \text{when } b \leq a$

2.3 3. Key Lattice Structures for Logic Programming

2.3.1 3.1 The Substitution Lattice

Unification operates on a lattice of substitutions:

- $\bot$ = identity substitution (no bindings)
- $\sigma \leq \tau$ iff τ is a specialization of σ (more bindings)
- $\sigma \sqcup \tau$ = most general unifier (mgu) of σ and τ
- $\top$ = failure (no unifier exists)

This is the fundamental lattice for Prolog-style logic programming: every resolution step computes a join on the substitution lattice.

2.3.2 3.2 The Herbrand Interpretation Lattice

For Datalog/bottom-up evaluation:

- Elements are sets of ground atoms (Herbrand interpretations)
- $\bot$ = empty set
- $I_1 \leq I_2$ iff $I_1 \subseteq I_2$
- $I_1 \sqcup I_2 = I_1 \cup I_2$
- The immediate consequence operator T_P maps an interpretation to its one-step derivation

The least fixed point of T_P gives the minimal Herbrand model — all provable ground facts.

2.3.3 3.3 The Interval Lattice

For numeric constraint propagation:

- Elements are intervals $[a, b]$ over $\mathbb{R}$
- $\bot = \mathbb{R}$ (no constraint)
- $[a_1, b_1] \sqcap [a_2, b_2] = [\max(a_1, a_2), \min(b_1, b_2)]$ (intersection = more constrained)
- $\top = \emptyset$ (contradictory constraints)

2.3.4 3.4 The Protocol State Lattice (Session Types)

For Prologos's session type verification:

- $\bot$ = unconstrained channel
- Interior elements = specific protocol states (e.g., "send Int then receive String")
- $\top$ = protocol violation
- Duality checking: two endpoints' protocol states must be lattice-dual

2.3.5 3.5 The Multiplicity Semiring (QTT)

For Prologos's quantitative type theory:

- Elements: $\{0, 1, \omega\}$ (erased, linear, unrestricted)
- Addition: resource usage accumulation
- Multiplication: usage in composed contexts
- This forms a *semiring*, not a lattice per se, but the ordering $0 \leq 1 \leq \omega$ creates a lattice structure useful for propagation

3 Part II: The Architectural Recommendation

3.1 4. The Three-Layer Architecture

We recommend a three-layer architecture for Prologos's logic engine:

```
Layer 3: STRATIFICATION
Negation, aggregation, non-monotone operations
Evaluated stratum-by-stratum (lower strata first)



Layer 2: ATMS / TRUTH MAINTENANCE
Hypothetical reasoning (amb, worldviews)
Nogood recording, dependency-directed backtracking
Non-monotonic control over monotonic data



Layer 1: PROPAGATOR NETWORK
Cells (lattice values) + Propagators (monotone fns)
Merge = lattice join
Deterministic, order-independent, parallelizable
```

3.1.1 Layer 1: The Propagator Network (Monotonic Data Plane)

The foundation. Cells hold partial information (lattice elements); propagators compute monotone functions between cells. When a cell's value increases

(via merge/join), all downstream propagators are scheduled for re-execution. This layer guarantees:

- **Determinism**: same inputs → same fixed point, regardless of scheduling
- **Convergence**: finite-height lattices terminate; widening handles infinite ones
- **Parallelism**: propagators can execute concurrently (CALM theorem)

This is the layer where type inference, constraint propagation, and dataflow computation live. It is purely monotonic.

3.1.2 Layer 2: ATMS / Truth Maintenance (Non-Monotonic Control Plane)

The innovation. The ATMS maintains *all possible worldviews simultaneously.* Each derived fact is labeled with the set of assumptions that justify it.

- `amb` creates fresh hypothetical premises: $\{h_1, h_2, \ldots\}$
- Each branch's facts are *contingent* on its hypothesis
- **Nogoods**: when a contradiction is detected, the responsible assumption set is recorded as a nogood — permanently invalidating that worldview
- **Dependency-directed backtracking**: nogoods identify *which* choice was wrong, skipping irrelevant alternatives

This layer provides nondeterministic search *without backtracking* in the traditional sense. All branches exist simultaneously in the ATMS; "exploring a branch" = believing a hypothesis; "backtracking" = recording a nogood.

Key insight from the research: **the ATMS's set of nogoods is itself a monotonically growing lattice**. Non-monotonicity in the search space is reified as monotonicity in the meta-lattice of learned constraints.

3.1.3 Layer 3: Stratification (Controlled Non-Monotonicity)

Some operations are genuinely non-monotonic: negation-as-failure, aggregation (`count`, `min`, `max`), and finalization. These are handled by **stratification**:

1. Decompose the program into strata (strongly connected components of the dependency graph, with non-monotonic edges crossing strata)
2. Evaluate strata bottom-up: within each stratum, run the propagator network to its fixed point
3. Between strata, non-monotonic operations observe the *completed* lower stratum

This is the established technique from Datalog with negation, Flix, Bloom^L, and CHR. The CALM theorem confirms: within a stratum, computation is coordination-free; stratum boundaries are the minimal coordination points.

3.2 5. Validating the Multiverse Mechanism

The user's proposed "Multiverse Mechanism" — forking lattice state at choice points, exploring branches independently — is *directly validated* by the research. It maps cleanly onto the ATMS worldview model:

Multiverse Concept	ATMS Implementation
Choice point	`amb` creating hypothetical premises $\{h_i\}$
Fork	Each h_i labels a worldview
Independent evolution	Facts derived under h_i are contingent
Branch contradiction	Nogood set $\{h_i, \ldots\}$ recorded
Branch success	Answer extracted from worldview h_i
All branches at once	ATMS maintains all worldviews simultaneously

3.2.1 Why ATMS over Physical Forking

The research reveals two approaches to forking:

1. **Physical forking** (MUSE/OR-parallelism): Copy the entire state at each choice point. Each copy evolves independently.
   - Pro: Simple, naturally parallel
   - Con: $O(\text{state}_{\text{size}})$ per fork; no shared learning between branches
2. **Virtual forking** (ATMS): Tag all facts with their supporting assumptions. All worldviews coexist in a single data structure.
   - Pro: O(1) context switching; nogoods shared across worldviews; dependency-directed backtracking

- Con: More complex bookkeeping; label management overhead

Recommendation for Prologos: ATMS (virtual forking). The reasons:

- Prologos already has contingent information in its type checker (metavariables with constraints that may be retracted)
- Dependency-directed backtracking produces dramatically better error messages ("this constraint fails because of *these* assumptions") vs chronological backtracking ("something went wrong somewhere")
- The learning effect: nogoods discovered in one branch prune other branches. This is exactly CDCL (Conflict-Driven Clause Learning), which revolutionized SAT solving
- Memory efficiency: shared structure between worldviews rather than full copies

3.2.2 The Multiverse in Practice: A Worked Example

```
;; A Prologos logic program (hypothetical syntax)
relation parent : String -> String -> Prop
parent "alice" "bob"
parent "bob" "carol"
parent "bob" "dave"

relation ancestor : String -> String -> Prop
rule ancestor X Y :- parent X Y
rule ancestor X Y :- parent X Z, ancestor Z Y

query ancestor "alice" ?who
;; Expected: ?who = "bob", ?who = "carol", ?who = "dave"
```

Under ATMS:

1. Create cell for `?who`, initially $\bot$
2. First rule: `parent "alice" ?who` unifies, producing `?who = "bob"` under assumption h_1
3. Second rule: `parent "alice" ?Z`, `ancestor ?Z ?who`

- `?Z = "bob"` under h_2; then `ancestor "bob" ?who`
- First rule: `parent "bob" ?who` $\rightarrow$ `?who = "carol"` under h_3 or `?who = "dave"` under h_4 (`amb` on matching clauses)

4. All answers coexist: `{h_1: "bob", h_3: "carol", h_4: "dave"}`

5. Collecting answers = iterating over consistent worldviews

3.3 6. Validating the Pocket Universe Mechanism

The "Pocket Universe Mechanism" — isolating a sub-computation in a smaller lattice, then projecting results back — maps onto **Galois connections** from abstract interpretation:

Pocket Universe Concept	Formal Mapping
Outer lattice (full)	Concrete domain C
Inner lattice (pocket)	Abstract domain A
Enter pocket	Abstraction function $\alpha : C \rightarrow A$
Compute in pocket	Fixed point in A (cheaper/faster)
Exit pocket	Concretization $\gamma : A \rightarrow C$
Soundness guarantee	Galois connection: $\alpha(x) \leq y \iff x \leq \gamma(y)$

3.3.1 Applications in the Logic Engine

1. **Tabling as pocket universes**: When a tabled predicate is called, a sub-computation is launched in a "pocket" (the table). Intermediate results are accumulated in the table's lattice. When complete, the table's answers are projected back into the calling context via the consumer mechanism.

2. **Modular constraint domains**: Each constraint domain (equality, ordering, arithmetic, set membership) lives in its own lattice. They are embedded into the global propagator network via Galois connections. Each domain solver is a "pocket universe" that receives projections of the global state and returns refined constraints.

3. **Scoped hypothetical reasoning**: An `amb` creates a pocket universe for each alternative. Within each pocket, propagation proceeds independently. Nogoods discovered in one pocket prune others via the ATMS label mechanism.

4. **Abstract interpretation for type checking**: The type checker operates in an abstract domain (types as lattice elements) connected to the concrete domain (values) via a Galois connection. This is already implicit in Prologos's bidirectional type checking; making it explicit with propagators would improve composability and error reporting.

3.4 7. Recovering SLD/SLG Semantics

3.4.1 7.1 SLD Resolution on Propagators

SLD resolution (Prolog's execution model) maps onto propagators as follows:

SLD Concept	Propagator Implementation
Goal stack	Worklist of pending propagator firings
Clause selection	`amb` over matching clauses
Unification	Join on the substitution lattice
New subgoals	New propagators added to the network
Backtracking	Nogood recording + hypothesis retraction
Answer	Fixed point of the substitution cell

The key difference: SLD's depth-first, left-to-right search becomes the ATMS's breadth-first, dependency-directed search. This is *more complete* (no infinite loops from left-recursion) and *more efficient* (nogoods prune the search space globally).

3.4.2 7.2 SLG Resolution (Tabling) on Propagators

SLG resolution extends SLD with memoization. The propagator mapping:

SLG Concept	Propagator Implementation
Table (memo store)	LVar with lattice answer mode
Producer	Propagator computing new answers
Consumer	Propagator waiting for table answers (threshold read)
Completion	Table LVar frozen; no more answers
Well-founded semantics	Stratified evaluation of cyclic dependencies

Lattice answer modes (from XSB Prolog): instead of storing all individual answers, the table stores the lattice join of all answers. This subsumes Datalog evaluation and connects directly to Flix's lattice-valued relations.

```
;; SLG-style tabling with lattice answer mode
;; shortest_path(A, B, D) - D is a MinNat lattice value
table shortest_path : String -> String -> MinNat -> Prop
  :answer-mode lattice  ;; join answers via min

rule shortest_path A B 1 :- edge A B
rule shortest_path A C .{D1 + D2}
  :- edge A B, shortest_path B C D2, .{D1 = 1}
```

3.4.3 7.3 Cut and Pruning

Cut maps onto ATMS operations:

Cut Variant	Propagator Implementation
`!` (hard cut)	Retract alternative hypotheses for this `amb`
`once/1`	Threshold read + freeze on first answer
`if-then-else`	Conditional propagator (threshold on condition)
Committed choice	Guard evaluation + monotone commitment
`\+` (negation-as-failure)	Stratified: evaluate goal in lower stratum, negate

The ATMS approach is superior to Prolog's chronological backtracking because cut's *dependency-directed* counterpart (nogood recording) naturally identifies *which* choices to prune, avoiding the well-known pitfalls of "red cuts" that silently change program semantics.

Recommendation: Prologos should support `once` (deterministic commit) and `committed-choice` (guard-based), but NOT raw `!` (Prolog-style cut). Cut is too low-level and error-prone; the ATMS provides better alternatives.

3.5 8. Tabling: The Completeness Mechanism

Tabling is *essential* for a practical logic engine. Without it, left-recursive rules cause infinite loops (in SLD) or redundant computation (in naive bottom-up evaluation).

3.5.1 8.1 Design for Prologos

```
;; Tabling declaration via spec metadata
spec ancestor : String -> String -> Prop
  :tabled true
```

```
  :answer-mode all  ;; or :answer-mode lattice for aggregate answers

;; Implementation is just regular rules
rule ancestor X Y :- parent X Y
rule ancestor X Y :- parent X Z, ancestor Z Y
```

Under the hood:

1. First call to `ancestor` creates a *producer* propagator
2. The producer evaluates rules, pushing answers into a table (LVar)
3. Subsequent calls create *consumer* propagators that read from the table via threshold reads
4. When the producer reaches a fixed point (no new answers), the table is frozen
5. Consumers receive all answers

3.5.2 8.2 Lattice Answer Modes

Following XSB Prolog's design, each tabled predicate can specify an answer mode:

- `all`: Store all distinct answers (standard tabling). The lattice is the powerset of substitutions.
- `lattice(f)`: Aggregate answers via a user-specified lattice join `f`. New answers are only "new" if they improve the current aggregate. This subsumes Datalog with lattices (Flix-style).
- `first`: Store only the first answer (`once` semantics, table frozen after first answer).

3.5.3 8.3 Tabling as Pocket Universe

Each tabled predicate's table is a "pocket universe" in the Galois connection sense:

- **Abstraction**: project the calling context's substitution onto the tabled predicate's argument positions
- **Computation**: evaluate the tabled predicate's rules within the pocket

- **Concretization**: lift the table's answers back into the calling context

This ensures that tabled sub-computations are modular and can be incrementally updated when the calling context provides new information.

3.6 9. Connecting to Existing Prologos Infrastructure

3.6.1 9.1 The Elaborator as a Propagator Network (Today)

Prologos's elaborator *already* implements propagator-like behavior:

Current Elaborator	Propagator Equivalent
Metavariables	Cells (lattice of types/terms)
Unification constraints	Propagators (join on subst lattice)
`resolve-trait-constraints!`	Propagators resolving dict params
`save/restore-meta-state!`	Speculative execution (worldviews)
`check-unresolved`	Detecting unsolved cells
Zonking	Reading final fixed-point state

The gap: these are *ad hoc* rather than *architected*. The metavar store is a mutable hash table, not a lattice-structured cell. Trait resolution uses a worklist, but doesn't record nogoods or do dependency-directed backtracking. Speculative execution (`save/restore-meta-state!`) copies the entire state, rather than using ATMS-style contingent values.

Phase 1 recommendation: Refactor the elaborator's metavar system to use proper propagator cells with lattice merge semantics. This would immediately improve:

- Error messages (dependency tracking shows *why* a constraint failed)
- Implicit argument inference (more information from bidirectional flow)
- Speculative type checking (ATMS worldviews instead of full state copy)

3.6.2 9.2 Session Types as Protocol Propagation

Session type checking already propagates protocol state. Making this explicit with propagator cells would enable:

- Multi-party session typing (currently only binary)
- Dependent session types (protocol depends on values)
- Better error messages for protocol violations

3.6.3 9.3 QTT Multiplicity Checking as Propagation

Multiplicity inference (Sprint 7) already propagates multiplicity constraints. With proper propagator cells:

- Multiplicity inference becomes a standard fixpoint computation
- Multiplicity errors get dependency tracking
- The semiring structure $(0, 1, \omega)$ is naturally a lattice

4 Part III: The Concrete Architecture

4.1 10. Core Data Types

4.1.1 10.1 Lattice Trait

```
;; The fundamental lattice abstraction
trait Lattice {A : Type}
  bot   : A
  join  : A -> A -> A
  leq   : A -> A -> Bool

;; Laws (checked via property testing):
;;   join is commutative:  join a b = join b a
;;   join is associative:  join (join a b) c = join a (join b c)
;;   join is idempotent:   join a a = a
;;   bot is identity:      join bot a = a
;;   leq is consistent:    leq a b = (join a b == b)
```

4.1.2 10.2 Propagator Cell

```
;; A cell holds a lattice value, tracks dependents
deftype Cell (A : Type)
  :where (Lattice A)
  mk-cell : A -> Cell A

;; Core operations
spec cell-read  : {A : Type} where (Lattice A) [Cell A] -> A
spec cell-write : {A : Type} where (Lattice A) [Cell A] -> A -> Unit
  ;; cell-write c v = merge(current, v); schedule dependents if changed
```

```
spec cell-watch : {A : Type} where (Lattice A) [Cell A] -> [A -> Unit] -> Unit
  ;; Register a callback (propagator) triggered on cell value change
```

4.1.3 10.3 Supported Values (ATMS Integration)

```
;; A supported value pairs information with its justification
deftype Supported (A : Type)
  mk-supported : A -> [Set Assumption] -> Supported A

;; A TMS cell holds multiple contingent values
deftype TMSCell (A : Type)
  :where (Lattice A)
  mk-tms-cell : [Set [Supported A]] -> TMSCell A
```

4.1.4 10.4 LVar (Lattice Variable)

```
;; LVar: monotonic lattice variable with threshold reads
deftype LVar (A : Type)
  :where (Lattice A)
  mk-lvar : A -> LVar A

spec lvar-put : {A : Type} where (Lattice A)
  [LVar A] -> A -> Unit
  ;; Inflationary: new_state = join(current, value)

spec lvar-get : {A : Type} where (Lattice A)
  [LVar A] -> [Set A] -> A
  ;; Threshold read: block until state >= some element of threshold set
  ;; Threshold elements must be pairwise incompatible

spec lvar-freeze : {A : Type} where (Lattice A)
  [LVar A] -> A
  ;; Freeze and return exact state (quasi-deterministic)
```

4.2 11. The Propagator Network Runtime

4.2.1 11.1 Scheduler

```
Scheduler
```

```
  Worklist (queue)
  [prop1, prop3, ...]


Loop:
  1. Dequeue propagator
  2. Execute (read inputs)
  3. Write outputs (merge)
  4. If cell changed:
     enqueue dependents
  5. If cell = top:
     contradiction handler
  6. Repeat until empty
```

Properties:

- Jobs are idempotent (safe to re-execute)
- Scheduling order doesn't affect the final result (lattice determinism)
- Parallelizable: independent propagators can execute concurrently

4.2.2 11.2 Contradiction Handler

When a cell reaches $\top$ (contradiction):

1. Extract the *support set* (assumptions that led to contradiction)
2. Record as a **nogood** in the ATMS
3. Retract the most recently chosen hypothesis in the nogood set
4. Re-propagate with the retracted hypothesis disbelieved

This is dependency-directed backtracking. It avoids the exponential blowup of chronological backtracking by targeting the actual cause.

4.2.3 11.3 The `amb` Operator

```
;; amb creates a choice point with n alternatives
spec amb : {A : Type} [List A] -> A
```

```
;; Under the hood:
;; 1. Create fresh hypotheses h1, h2, ..., hn
;; 2. For each hi, create supported value (ai, {hi})
;; 3. Add mutual exclusion: exactly one hi is believed
;; 4. Return the cell containing the contingent value
```

4.2.4 11.4 Answer Collection

```
;; Collect all solutions from a nondeterministic computation
spec solve-all : {A : Type} [Unit -> A] -> [List A]

;; Under the hood:
;; 1. Run the computation with ATMS
;; 2. Iterate over all consistent worldviews at quiescence
;; 3. For each worldview, extract the answer
;; 4. Return list of all answers (duplicates removed)
```

4.3 12. Phased Implementation Plan

4.3.1 Phase 0: Lattice Trait + Basic Cells (Foundation)

Goal: Establish the lattice trait and basic propagator cell infrastructure at the Racket level.

- Define `Lattice` trait in Prologos
- Provide instances: `FlatLattice` (bottom/value/top), `SetLattice` (powerset with union), `IntervalLattice`
- Implement `Cell` as a Racket-level struct with mutable value and dependent list
- Implement basic scheduler (worklist, fire-till-quiescence)
- **~15 AST nodes** (cell-new, cell-read, cell-write, cell-watch, ...)
- **~40 tests**

Dependencies: None (builds on existing trait system) **Estimated effort**: Medium

4.3.2 Phase 1: Elaborator Propagators (Compile-Time Use)

Goal: Refactor the metavar system to use propagator cells internally.

- Replace `current-meta-store` with propagator cells
- Unification constraints become propagators between type cells
- Trait resolution constraints become propagators
- Add dependency tracking to metavar refinements
- Improve error messages using dependency information
- **No new surface syntax** — this is internal refactoring
- **~60 tests** (regression + new dependency-tracking tests)

Dependencies: Phase 0 **Estimated effort**: Large (touches elaborator + typing-core + unify) **Key risk**: This is a significant refactoring of the elaborator core

4.3.3 Phase 2: LVars + Tabling (Runtime Foundation)

Goal: LVar data structures and tabled evaluation.

- Implement `LVar` with `put`, threshold `get`, and `freeze`
- Implement `LVar-Set`, `LVar-Map` (grow-only collections)
- Implement tabling framework: producer/consumer pattern, table LVars, completion detection
- Add `:tabled` spec metadata handling
- **~20 AST nodes** (lvar-new, lvar-put, lvar-get, lvar-freeze, table-lookup, ...)
- **~50 tests**

Dependencies: Phase 0, Phase 2d of Core DS roadmap (transient builders) **Estimated effort**: Large

4.3.4 Phase 3: ATMS Layer (Nondeterministic Search)

Goal: Assumption-based truth maintenance for hypothetical reasoning.

- Implement `Assumption`, `Supported`, `TMSCell` types
- Implement nogood recording and management
- Implement `amb` operator with hypothesis creation
- Implement contradiction handler with dependency-directed backtracking
- Implement `solve-all` answer collection
- **~15 AST nodes** (amb, assume, retract, nogood, ...)
- **~60 tests**

Dependencies: Phase 0, Phase 2 **Estimated effort**: Large (most complex phase)

4.3.5 Phase 4: Stratified Evaluation (Negation + Aggregation)

Goal: Support negation-as-failure and aggregation via stratification.

- Implement SCC decomposition of rule dependency graphs
- Implement stratum-by-stratum evaluation
- Support `not` (negation-as-failure) between strata
- Support lattice aggregation (`count`, `min`, `max`, `sum`) between strata
- **~10 AST nodes**
- **~40 tests**

Dependencies: Phase 3 **Estimated effort**: Medium

4.3.6 Phase 5: Surface Syntax + Integration (User-Facing Logic Programming)

Goal: Prologos surface syntax for logic programming.

- `relation` declarations
- `rule` and `query` syntax
- Pattern matching integration with unification
- `defn` bodies that can contain logic variables
- Integration with the prelude and module system
- **Grammar updates** (grammar.ebnf, grammar.org)
- **~80 tests**

Dependencies: All previous phases **Estimated effort**: Large

4.3.7 Phase 6: Galois Connections + Domain Embeddings (Advanced)

Goal: Modular constraint domains connected via Galois connections.

- `domain` declarations with abstraction/concretization functions
- Cross-domain propagation
- Abstract interpretation framework for program analysis
- **~30 tests**

Dependencies: Phase 5 **Estimated effort**: Medium-Large

5 Part IV: Critique and Open Questions

5.1 13. What This Architecture Does Well

1. **Unification of paradigms**: The same propagator substrate serves type checking, constraint solving, and logic programming. This is not merely aesthetic — it means improvements to the propagator infrastructure benefit all three use cases.

2. **Sound theoretical basis**: Every component has well-understood lattice-theoretic foundations. Convergence, determinism, and soundness are not hoped for but *proven* by the mathematical framework.

3. **Incremental by construction**: Propagator networks naturally support incremental computation. When a constraint is added or retracted, only affected cells re-propagate. This is essential for interactive development (IDE integration, REPL).

4. **Parallelizable by the CALM theorem**: Within a stratum, all propagation is monotonic and therefore coordination-free. This provides a natural path to parallel type checking and parallel logic evaluation.

5. **Better error messages**: Dependency tracking in the ATMS means every derived fact carries its provenance. Type errors can show exactly *which* constraints are in tension and *where* they originated.

5.2 14. What This Architecture Doesn't Solve (Yet)

1. **Performance**: ATMS label management has overhead. For simple deterministic programs, the ATMS layer is pure cost. We need a *bypass* for deterministic code paths that avoids the TMS entirely.

2. **Infinite domains**: Widening operators are needed for lattices of infinite height (intervals, numeric constraints). Choosing the right widening strategy is domain-specific and sometimes tricky.

3. **Efficiency of lattice answer modes**: When the lattice is large or the join is expensive, tabling with lattice answer modes can be slow. XSB Prolog's experience shows this requires careful implementation.

4. **Surface syntax design**: How should logic programming syntax look in Prologos? The document proposes `relation` / `rule` / `query` but this needs design iteration. Should it integrate with `match`? With `where` constraints? With mixfix `.{...}`?

5. **Interaction with QTT**: Logic variables are inherently shared (multiple references). How does this interact with linear types? Preliminary answer: logic variables live at multiplicity ω (unrestricted), but the *binding environment* can be linear.

5.3 15. Open Research Questions

1. **Can ATMS label management be made zero-cost for deterministic programs?** The "speculative ATMS" idea: don't create labels until the first `amb`. Deterministic programs never pay the ATMS overhead.

2. **How do Prologos's dependent types interact with tabling?** Tabling requires comparing calls for "variance" (same query up to renaming). With dependent types, this comparison involves type equality, which may itself require propagation. Circularity risk.

3. **Can propagator-based elaboration subsume the current ad-hoc metavar system without performance regression?** The current system is fast because it's specialized. A general propagator system may be slower without careful optimization.

4. **What is the right granularity for stratification?** Per-predicate? Per-SCC? Per-module? Finer granularity allows more parallelism but increases analysis cost.

5. **How do pocket universes compose?** Can a tabled predicate call another tabled predicate? (Yes, but completion detection becomes more complex — this is the "scheduling dependency" problem in SLG resolution.)

6 Summary of Recommendations

#	Recommendation	Confidence
1	Radul-Sussman propagator model as the substrate	High
2	ATMS for hypothetical reasoning (validates Multiverse Mech.)	High
3	Galois connections for domain modularity (validates Pocket U.)	High
4	SLG-style tabling with lattice answer modes	High
5	Stratified evaluation for negation and aggregation	High
6	Start with compile-time use (elaborator), then extend to runtime	High
7	No raw Prolog-style `!` cut; use `once` and committed choice	Medium
8	"Speculative ATMS" for zero-cost deterministic bypass	Medium
9	`Lattice` trait as foundation (user-extensible domains)	High
10	Surface syntax: `relation` / `rule` / `query` keywords	Low (needs design)

Overall assessment: The research strongly supports the proposed architecture. The Multiverse and Pocket Universe mechanisms are not speculative — they are well-established techniques (ATMS worldviews and Galois connections respectively) with decades of theoretical grounding and practical implementation experience. The main risk is implementation complexity, mitigated by the phased approach starting with the elaborator refactoring.

7 References

Organized by relevance to the recommended architecture:

7.1 Foundational (Must-Read)

- Radul & Sussman, "The Art of the Propagator" (MIT TR, 2009)
- Radul, *Propagation Networks* (PhD thesis, MIT, 2009)
- de Kleer, "An Assumption-Based TMS" (*AI Journal*, 1986)
- Kuper & Newton, "LVars: Lattice-Based Data Structures for Deterministic Parallelism" (FHPC, 2013)
- Kuper et al., "Freeze After Writing" (POPL, 2014)

7.2 Tabling and Logic Programming

- Swift & Warren, "XSB: Extending Prolog with Tabled Logic Programming" (*TPLP*, 2012)
- Chen & Warren, "Tabled Evaluation with Delaying" (*JACM*, 1996)
- Gupta et al., "Parallel Execution of Prolog Programs: A Survey" (*TOPLAS*, 2001)

7.3 Lattice-Based Languages

- Madsen et al., "From Datalog to Flix" (PLDI, 2016)
- Arntzenius & Krishnaswami, "Datafun: A Functional Datalog" (ICFP, 2016)
- Arntzenius & Krishnaswami, "Seminaive Evaluation for a Higher-Order Functional Language" (POPL, 2020)

7.4 Theoretical Foundations

- Tarski, "A Lattice-Theoretical Fixpoint Theorem" (*Pacific J. Math.*, 1955)
- Cousot & Cousot, "Abstract Interpretation: A Unified Lattice Model" (POPL, 1977)
- Denecker et al., "Approximation Fixpoint Theory" (2012)
- Hellerstein, "Keeping CALM" (*CACM*, 2020)

7.5 Constraint Systems

- Fruhwirth, *Constraint Handling Rules* (CUP, 2009)
- Shapiro et al., "Conflict-Free Replicated Data Types" (INRIA, 2011)
- Conway et al., "Logic and Lattices for Distributed Programming" (SoCC, 2012)

7.6 Type Systems

- Dunfield & Krishnaswami, "Bidirectional Typechecking for Higher-Rank Polymorphism" (ICFP, 2014)
- Pottier & Remy, "The Essence of ML Type Inference" (2005)
- Kovacs, *Elaboration Zoo* (2022)

7.7 Implementations

- Holmes (Haskell): `github.com/i-am-tom/holmes`
- Propaganda (Clojure): `github.com/tgk/propaganda`
- Ascent (Rust): `github.com/s-arash/ascent`
- Guanxi (Haskell): `github.com/ekmett/guanxi`